



Universidad Nacional de Rosario
Facultad de Ciencias Exactas, Ingeniería y Agrimensura
Escuela de Ingeniería Electrónica
Proyecto de Ingeniería

Simulación de la topología Matriz de Conmutación mediante Hardware In Loop (HIL)

Autor: Combin Lucas Nahuel (C-6630/3)
Director: Ing. Sergio Geninatti
Asesores: Ing. Mónica Romero, Dra. María M. Serón

Octubre 2025

Resumen

El presente proyecto tiene como objetivo el diseño e implementación de un entorno de simulación Hardware in Loop (HIL) para la topología de convertidor AC-AC denominada Matriz de Conmutación, controlada mediante la estrategia de Modulación por Vector Espacial (SVM). Dado que estos sistemas operan con potencias elevadas, la metodología HIL permite realizar pruebas y validar el comportamiento del sistema sin necesidad de hardware físico de potencia, reduciendo costos y riesgos asociados.

El desarrollo se llevó a cabo sobre una FPGA Zynq-7000 (placa BlackBoard), utilizando VHDL como lenguaje de descripción de hardware y Python como herramienta de validación. Se diseñaron e implementaron los bloques constitutivos del sistema: fuente de tensión trifásica, Matriz de Conmutación, carga RL trifásica, Transformada de Clarke y generadores de referencias angulares mediante diente de sierra y algoritmo CORDIC.

Los resultados obtenidos demuestran que el sistema es capaz de generar corrientes trifásicas sinusoidales filtradas y un vector espacial de corriente rotante correcto, validando la metodología HIL propuesta. Se identificaron y resolvieron desafíos en aritmética de punto fijo, manejo de signo en complemento a 2, y problemas de timing en la implementación física, los cuales fueron solucionados mediante técnicas de pipelining, optimización lógica y diseño multi-clock.

Índice

	Página
1. Introducción	3
1.1. Problemática	3
1.2. Consideraciones previas	3
2. Propuesta de Trabajo	5
2.1. Dispositivo a utilizar	5
2.2. Software a utilizar	5
2.3. Pautas de Diseño	5
2.4. Metodología de Desarrollo	5
2.4.1. Fase I: Investigación de la tecnología	5
2.4.2. Fase II: Desarrollo de los diferentes bloques	6
2.4.3. Fase III: Simulaciones	6
2.4.4. Fase III: Implementación estructural del sistema	6
3. Desarrollo de Componentes Base	7
3.1. Modulador	7
3.2. Fuente de Tensión Trifásica	8
3.3. Matriz de Conmutación	9
3.4. Carga RL	11
3.5. Space Vector Modulator - SVM	13
3.6. Transformada de Clarke	14
3.7. Generador de referencias angulares	15
3.7.1. Generador Diente de Sierra	15
3.7.2. CORDIC - atan2	16
4. Simulaciones	19
4.1. Fuente de señal trifásica y Transformada de Clarke	19
4.2. Generador Alfa y Beta	19
4.2.1. Generador Diente de Sierra	19
4.2.2. CORDIC - atan2	20
4.3. Carga RL trifásica	21
4.4. Modulador SVM	22
4.4.1. Análisis interno	23
4.4.2. Solución	24
5. Implementación en FPGA	26
5.1. Síntesis	26
5.2. Implementación	26
5.2.1. Problemas de Timing	26
5.2.2. Soluciones	26
5.2.3. Conclusion General	27
6. Comentarios Finales	28
6.1. Conclusiones	28

6.1.1.	Validación de la Metodología HIL (Hardware In Loop)	28
6.1.2.	Desafíos en Aritmética de Punto Fijo y VHDL	28
6.1.3.	Implementación en FPGA y Análisis de Timing	28
6.1.4.	Arquitectura y Herramientas	28
6.1.5.	Conclusion Final	28
6.2.	Posibles Mejoras	29
A.	Conversor Matricial - SVM	30
A.1.	Principio Fundamental del Algoritmo SVM	30
A.2.	Definición y Propósito de los Parámetros de Control	31

1. Introducción

La modulación por Matriz de Conmutación es una topología de convertidor electrónico basado en la conmutación de trifásica a trifásica (AC-AC). Existen diferentes estrategias de conmutación, entre ellas se encuentran la modulación por ancho de pulso (MDCM) o la modulación por espacio vectorial (SVM).[1]

Esta no es una tecnología nueva, ya que se comenzó a estudiar en la década del 70, pero recién en los últimos años se ha comenzado a utilizar en aplicaciones comerciales, debido a la mejora en los componentes electrónicos de potencia, como los IGBTs o los MOSFETs, que permiten conmutar a altas frecuencias y con bajas pérdidas. Además de dispositivos de control más rápidos y potentes, como los DSPs o los FPGAs.

1.1. Problemática

Debido a que estos sistemas trabajan con potencias elevadas, los costos de implementación son altos y conviene tener una forma de realizar pruebas y validar los sistemas antes de implementarlos en hardware real. Por lo que se propone la simulación de la topología mediante Hardware in Loop (HIL). Con esto se podrá realizar diferentes pruebas y mediciones, sin la necesidad de un hardware físico, además de que se podrán testear diferentes métodos de control del sistema.

Por lo tanto, en el presente proyecto se realizará el diseño del ambiente para la simulación y el control de la topología **Space-Vector Modulation (SVM)**, según como está indicado en la figura Figure 1.

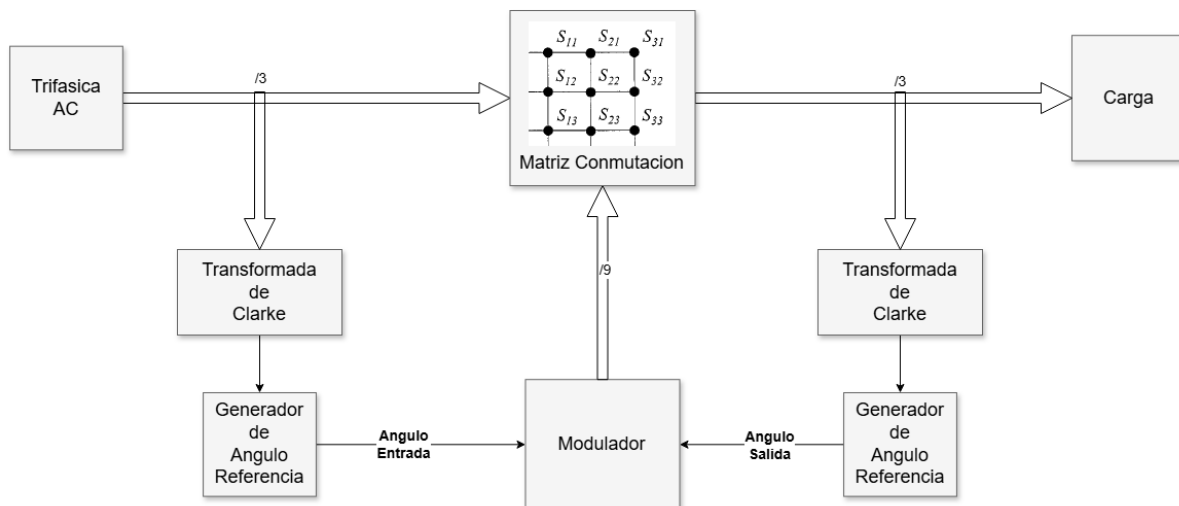


Figura 1. Sistema simulado

1.2. Consideraciones previas

Lo que se mencionó anteriormente como **Space-Vector Modulation (SVM)**, consiste en el conjunto Modulador + Matriz de Conmutación. El **Modulador** se encargará de generar las señales de control para la matriz de conmutación y la **Matriz de Conmutación** representa el arreglo de llaves conmutadoras que permiten la transferencia de energía entre la fuente y la carga.

Hay que tener en cuenta que el bloque **Modulador** se basa en un algoritmo encargado del calculo de los PWM para las llaves conmutadores de la matriz. Este algoritmo fue diseñado por la Dra. María M. Serón, y posteriormente gracia a la ayuda de la Ing. Mónica Romero que realizo un modelo en Matlab fue implementado en VHDL por el Ing. Sergio Geninatti. Es por esto que este desarrollo inicialmente se realizara en una FPGA.

Todo los códigos de este proyecto se encuentran en el siguiente repositorio de GitHub:

https://github.com/lucasCmb-eie/FinalProject_Ing

El proyecto tiene la siguiente estructura de carpetas:

```
/
├── HW/ ... Contiene todo el desarrollo en VHDL
│   │   para la implementación en la FPGA.
│   ├── src/ ... Contiene los archivos fuente del
│   │   │   proyecto.
│   │   ├── bd/ ... Contiene los archivos del bloque de
│   │   │   │   diseño (Block Design) de Vivado.
│   │   ├── constrs/ ... Contiene los archivos de
│   │   │   │   restricciones (XDC) del proyecto.
│   │   ├── hdl/ ... Contiene los archivos VHDL del
│   │   │   │   proyecto.
│   │   ├── ip/ ... Contiene los archivos de los bloques
│   │   │   │   IP personalizados.
│   │   ├── tb/ ... Contiene los archivos de los bancos
│   │   │   │   de prueba (testbenchs).
│   │   └── wrappers/ ... Contiene los archivos wrapper
│   │       │   generados por Vivado (.xsa).
├── SW/ ... Contiene todo el desarrollo que no
│   │   sea para la FPGA.
└── python/ ... Contiene los scripts en Python para
    │   generación rápida de datos.
```

2. Propuesta de Trabajo

2.1. Dispositivo a utilizar

Mínimamente el dispositivo a utilizar debe contar con una FPGA, ya que el modulador (la parte mas importante) esta diseñado en VHDL. Hoy en día existen diferentes opciones de dispositivos, desde los mas básicos como los de la línea Spartan-6, hasta los mas avanzados como los de la línea Virtex-7, todos de Xilinx.

Por la complejidad del proyecto, se opto por utilizar un dispositivo de gama media, como lo es la Zynq-7000 de Xilinx, la cual posee una Artix-7 con una buena cantidad de recursos para el desarrollo del proyecto, y a su vez es bastante económica.

Luego de ver varias opciones y disponibilidad de stock, se opto por la placa de desarrollo BlackBoard, que esta basada en la version mas básica del Zynq-7000, el Z-7010. Este dispositivo posee un SoC (System on Chip) que contiene un microcontrolador ARM Cortex-A9 (Single-Core), y una FPGA Artix-7.

2.2. Software a utilizar

Para el diseño del sistema dentro de la FPGA se utilizara la herramienta **Vivado** provista por Xilinx (no es necesario licencia paga para utilizarlas en este dispositivo), y Python para la validación de modelos y visualización de resultados.

2.3. Pautas de Diseño

Valores normalizados de Tensión Las amplitudes de las tensiones trifasicas $\in [-1V; 1V]$, cuando se tengan que extraer para su análisis o visualización se los va a acomodar a los valores reales correspondientes, así utilizamos menos cantidad de bits del formato seleccionado para representar números enteros.

Se trabajara con el formato Q8.24 Significa que se utilizara Punto Fijo, complemento 2, 8 bits para la parte entera (1 bit de signo, 7 bit de enteros) y 24 bits para la parte decimal, teniendo una resolución de $2^{-24} \approx 5,96 * 10^{-8}$.

Este formato nos permite representar números en el rango de $[-128 ; 127.99999994]$, que es mas que suficiente para los valores con los que se va a trabajar, y a su vez nos da una muy buena resolución.

2.4. Metodología de Desarrollo

El proyecto se estructurará en cuatro fases principales:

2.4.1. Fase I: Investigación de la tecnología

En esta fase se realizara una investigación sobre la tecnología a utilizar, dando énfasis en el significado de los parámetros de entrada que tiene el modulador[1].

Se estudiaran los diferentes bloques que componen el sistema, y se analizara su funcionamiento y su implementación en VHDL.

2.4.2. Fase II: Desarrollo de los diferentes bloques

Inicialmente se realizara el diseño de los diferentes bloques faltantes del sistema:

- Fuente de tensión trifásica
- Matriz de Conmutación
- Carga RL Trifásica

2.4.3. Fase III: Simulaciones

Con todos los bloques diseñados, se realizara la simulación de cada bloque y luego del sistema completo, para verificar los correctos funcionamientos.

En esta fase se prestara especial atención a la salida trifasica de corriente y a la salida del SVM, para analizar los resultados obtenidos.

De ser necesario, se realizaran ajustes en los bloques para mejorar el funcionamiento del sistema, y se volverá a simular hasta obtener los resultados deseados.

2.4.4. Fase III: Implementación estructural del sistema

Finalmente se realiza el proceso de síntesis e implementación, provistos por Vivado, esto es para verificar que el diseño es sintetizable (se puede "traducir.^a compuertas lógicas), y cumple con los requisitos de timing establecidos (las señales llegan a tiempo a cada bloque).

3. Desarrollo de Componentes Base

3.1. Modulador



Figura 2. Bloque del Modulador

Nombre	Tipo	Descripción
i_clk	std_logic	Señal de Reloj de 100MHz
i_enable	std_logic	Habilitar bloque
i_al_o	std_logic_vector(10:0)	Angulo de tensión de entrada α
i_be_i	std_logic_vector(10:0)	Angulo de corriente de salida β
i_q_i	std_logic_vector(8:0)	Factor de relación tensión entrada/salida
i_phi_i	std_logic_vector(10:0)	Angulo ϕ
o_direcciones	std_logic_vector(17:0)	Vector de tiempos de las llaves
o_fin_ciclo	std_logic	Pin de control
o_inicio_ciclo	std_logic	Pin de control
o_fin_calc_ts	std_logic	Pin de control

Tabla 1. Descripción de puertos

Este bloque puede tener un capítulo dedicado, ya que es el encargado de realizar los cálculos para obtener los PWM que controlaran las llaves conmutadoras de la matriz.

En esta sección se hará una breve descripción de su funcionamiento y como se implementó en VHDL. Para mas información se puede consultar el **Appendix A**.

Se representara como un bloque que posee 4 parámetros de entrada importantes: α , β , ϕ , q , que dependiendo de sus valores, se obtendrá un PWM diferente a la salida.

Los parámetros α , β son dinámicos, esto es así ya que representan los ángulos de la tensión de entrada y corriente de salida respectivamente que se desea tener en un instante de calculo.

Mientras que los parámetros ϕ , q , son referencias fijas, y representa el $\cos\phi$ que se desea que haya en la entrada, y $q = \frac{V_o}{V_i}$ es el factor de transferencia de tensión deseado. [1]

La salida de este bloque va a ser un vector de 13 elementos, solo utilizare los 9 primeros (del puerto 0 al 8), cada uno identifica el estado de una llave conmutadora, que ademas representan los coeficientes **M** del bloque **Matriz de Conmutación**.

3.2. Fuente de Tensión Trifasica

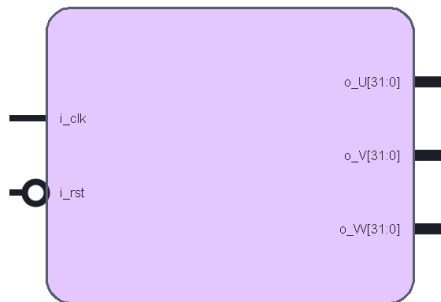


Figura 3. Bloque de la Fuente Trifasica

Nombre	Tipo	Descripción
i_clk	std_logic	Señal de Reloj de 100KHz
i_rst	std_logic	Pin de Reset
o_U	std_logic_vector(31:0)	Tensión de fase U
o_V	std_logic_vector(31:0)	Tensión de fase V
o_W	std_logic_vector(31:0)	Tensión de fase W

Tabla 2. Descripción de puertos

Este bloque simularía la alimentación de la linea, mediante la combinación de una LUT y un NCO. [2]

Lo que debe cumplir este bloque lo siguiente:

- Generar una señal trifásica de tensión senoidal, cada salida con 120° de desfase entre cada una.
- La frecuencia de la señal debe ser de 50Hz con una resolución de al menos 0.1 %, para estar lo mas cercano a la realidad.
- La amplitud de la señal $\in [-1V ; 1V]$, como se comento anteriormente se trabajara con tensiones normalizadas.
- Formato de datos de la salida en Q8.24.

Con estos puntos en mente, se decidió fijar que la frecuencia de reloj necesaria sea de 100KHz, ya que es una frecuencia que permite cumplir con la resolución deseada.

$$f_{res} = \frac{f_{clk}}{2^N} \quad (1)$$

Siendo N la cantidad de bits del NCO, buscando una resolución mejor que 0.05Hz, eligiendo N=24, obtenemos:

$$f_{res} = \frac{100000}{2^{24}} = 0,00596Hz \quad (2)$$

Que cumple con la resolución deseada.

Luego es necesario calcular la FTW (Frequency Tuning Word) que es el valor que se le debe cargar al NCO para que genere la frecuencia deseada.

$$FTW = \frac{f_{out} * 2^N}{f_{clk}} = \frac{50 * 2^{24}}{100000} \cong 8388,608 \quad (3)$$

Redondeando a al entero mas cercano, se obtiene FTW = 8389.

Finalmente se la frecuencia generada sera de:

$$f_{out} = \frac{FTW * f_{clk}}{2^N} = \frac{8389 * 100000}{2^{24}} \cong 50,0003Hz \quad (4)$$

Que es una frecuencia suficientemente cercana a la deseada. El error es de aproximadamente 0.001 %, cumpliendo con el requisito puesto.

Para el diseño de la LUT, usar los 24 bits del NCO es demasiado, ya que se tendrían 16777216 posiciones (2^{24}). Por lo que se decide usar los bits mas significativos MSB, como dirección. Entonces, el numero de muestras por ciclo serán:

$$Muestras = \frac{f_{clk}}{f_{out}} = \frac{100000}{50} = 2000 \quad (5)$$

Utilizando un valor de LUT próximo a este valor y que se a potencia de 2 es 2048 muestras. Por lo tanto se utilizara los M = 11 bits mas significativos del NCO para direccionar la LUT.

Para generar la LUT se utilizo un script en Python que genera un archivo de tipo package de VHDL con los valores en formato hexadecimal, que luego se convierten a binario para ser utilizados.

3.3. Matriz de Conmutación



Figura 4. Bloque de la Matriz de Conmutación

Nombre	Tipo	Descripción
i_clk	std_logic	Señal de Reloj de 100MHz
i_rst	std_logic	Pin de Reset
i_M	std_logic_vector(17:0)	Vector de PWM de las llaves
i_U	std_logic_vector(31:0)	Entrada de tensión de fase U
i_V	std_logic_vector(31:0)	Entrada de tensión de fase V
i_W	std_logic_vector(31:0)	Entrada de tensión de fase W
o_U	std_logic_vector(31:0)	Salida de tensión de fase U
o_V	std_logic_vector(31:0)	Salida de tensión de fase V
o_W	std_logic_vector(31:0)	Salida de tensión de fase W

Tabla 3. Descripción de puertos

Lo que hace la matriz de conmutación no es mas que permitir la conducción de las fases en las que las llave correspondiente esté cerradas y bloquear las que estén con la llave abierta. Por lo tanto se puede expresar de la siguiente manera:

$$\mathbf{V}_{out} = \mathbf{M} * \mathbf{V}_{in} \quad (6)$$

$$\begin{bmatrix} V_{Ou} \\ V_{Ov} \\ V_{Ow} \end{bmatrix} = \begin{bmatrix} m_{11} & m_{12} & m_{13} \\ m_{21} & m_{22} & m_{23} \\ m_{31} & m_{32} & m_{33} \end{bmatrix} * \begin{bmatrix} V_{Su} \\ V_{Sv} \\ V_{Sw} \end{bmatrix} \quad (7)$$

Siendo los coeficientes de $\mathbf{M}$: $m_{i,j}$ la representación de la llave conmutadora, y tendrán valores:

1 $\longrightarrow$ Llave cerrada

0 $\longrightarrow$ Llave abierta

Este producto matricial se puede reescribir como:

$$V_{outU} = m_{11} * V_{Su} + m_{12} * V_{Sv} + m_{13} * V_{Sw} \quad (8)$$

$$V_{outV} = m_{21} * V_{Su} + m_{22} * V_{Sv} + m_{23} * V_{Sw} \quad (9)$$

$$V_{outW} = m_{31} * V_{Su} + m_{32} * V_{Sv} + m_{33} * V_{Sw} \quad (10)$$

Este calculo se va a realizar en cada pulso del reloj con los valores que se posean tanto en la entrada como en los coeficientes de la matriz.

3.4. Carga RL

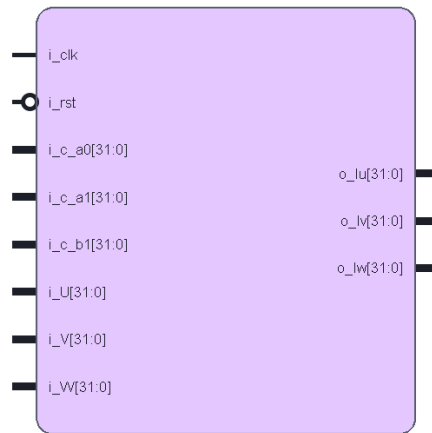


Figura 5. Bloque de la Carga RL trifásica

Nombre	Tipo	Descripción
i_clk	std_logic	Señal de Reloj de 100MHz
i_rst	std_logic	Pin de Reset
i_c_a0	std_logic_vector(31:0)	Coeficiente a_0 del filtro IIR
i_c_a1	std_logic_vector(31:0)	Coeficiente a_1 del filtro IIR
i_c_b1	std_logic_vector(31:0)	Coeficiente b_0 del filtro IIR
i_U	std_logic_vector(31:0)	Tensión de Entrada de fase U
i_V	std_logic_vector(31:0)	Tensión de Entrada de fase V
i_W	std_logic_vector(31:0)	Tensión de Entrada de fase W
o_U	std_logic_vector(31:0)	Corriente de Salida de fase U
o_V	std_logic_vector(31:0)	Corriente de Salida de fase V
o_W	std_logic_vector(31:0)	Corriente de Salida de fase W

Tabla 4. Descripción de puertos

Para el modelado de este tipo de carga utilizaremos las ecuaciones de los componentes:

$$i_L(t) = i_R(t) = i(t) \quad (11)$$

$$v_o(t) = i(t) * R + L \frac{di(t)}{dt} \quad (12)$$

Aplicando la transformada de Laplace, y asumiendo condiciones iniciales nulas obtenemos:

$$V_o(s) = I(s) * R + L * s * I(s) \quad (13)$$

Como vamos a controlar el sistema mediante la corriente en la carga, buscamos una función transferencia que relacione la corriente con la tensión de salida.

Para eso usaremos como función transferencia la Admitancia:

$$H(s) = \frac{I(s)}{V_o(s)} = \frac{1}{sL + R} \quad (14)$$

Se puede observar que es la función transferencia de un filtro paso bajo de primer orden. Con un polo en $s = -\frac{R}{L}$.

Para la implementación en VHDL, primero se debe discretizar. Utilizando la Transformación Bilineal, que es una Transformación precisa y mantiene la estabilidad del sistema.

Utilizando la expresión de la variable s , con T es el periodo de muestreo:

$$s \cong \frac{2}{T} \frac{1 - z^{-1}}{1 + z^{-1}} \quad (15)$$

Reemplazando en la función transferencia:

$$H(z) = \frac{I(z)}{V_o(z)} = \frac{1}{\frac{2L}{T} \frac{1 - z^{-1}}{1 + z^{-1}} + R} \quad (16)$$

$$= \frac{1 + z^{-1}}{\frac{2L}{T}(1 - z^{-1}) + R(1 + z^{-1})} \quad (17)$$

$$= \frac{1 + z^{-1}}{\left(\frac{2L}{T} + R\right) + \left(R - \frac{2L}{T}\right) z^{-1}} \quad (18)$$

Luego, se normaliza la expresión, dividiendo numerador y denominador por $\left(\frac{2L}{T} + R\right)$ para llegar a la expresión estándar de un filtro digital IIR:

$$H(z) = \frac{\frac{1}{\frac{2L}{T} + R}(1 + z^{-1})}{1 + \frac{R - \frac{2L}{T}}{R + \frac{2L}{T}} z^{-1}} \quad (19)$$

Esto nos da la ecuación en diferencias, que es la que se implementara en VHDL:

$$I(z) * \left(1 + \frac{R - \frac{2L}{T}}{R + \frac{2L}{T}} z^{-1}\right) = V(z) * \left(\frac{1}{\frac{2L}{T} + R} * (1 + z^{-1})\right) \quad (20)$$

$$I(z) + \left(\frac{R - \frac{2L}{T}}{R + \frac{2L}{T}} z^{-1}\right) = (V(z) + V(z)z^{-1}) * \left(\frac{1}{\frac{2L}{T} + R}\right) \quad (21)$$

$$I(z) = \left(\frac{1}{\frac{2L}{T} + R}\right) * (V(z) + V(z)z^{-1}) - \left(\frac{R - \frac{2L}{T}}{R + \frac{2L}{T}} z^{-1}\right) * I(z)z^{-1} \quad (22)$$

$$(23)$$

Finalmente de esta ultima expresión y aplicando la antitransformada de Z , se obtiene la ecuación en diferencias que define el comportamiento de la carga RL:

$$i(n) = a_0 * v(n) + a_1 * v(n - 1) + b_1 * i(n - 1) \quad (24)$$

Con:

$$a_0 = \frac{1}{\frac{2L}{T} + R} \quad (25)$$

$$a_1 = a_0 \quad (26)$$

$$b_1 = \frac{R - \frac{2L}{T}}{R + \frac{2L}{T}} \quad (27)$$

Algo a tener en cuenta, es que esta ecuación modela una única fase, por lo que, en la implementación se repetirá 3 veces este código para representar cada fase.

Los coeficientes a_0 , a_1 y b_1 van a ser entradas del bloque, para poder variarlas desde el exterior, y además van a ser compartidas entre las 3 fases (se supone con esto carga balanceada).

3.5. Space Vector Modulator - SVM

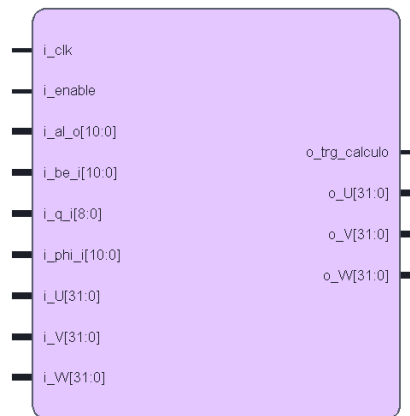


Figura 6. Bloque del SVM

Es la combinación física de los bloques de la Matriz de Conmutación y el Modulador, por lo tanto este bloque se encarga tanto de calcular los PWM de las llaves conmutadoras, como de realizar la conmutación.

La única particularidad que tiene es que posee una salida $o_trigger$, que se activa cada vez que comienza un nuevo ciclo de PWM, y se utiliza para sincronizar el cálculo de la transformada de Clarke, y así obtener los valores de α y β para el siguiente ciclo.

En la implementación real, esta señal dispara un ADC para obtener los valores de tensión y corriente de entrada, y así comenzar el flujo de cálculo de los valores de ángulos necesario.

3.6. Transformada de Clarke



Figura 7. Bloque de la Transformada de Clarke

Nombre	Tipo	Descripción
i_clk	std_logic	Señal de Reloj de 100MHz
i_rst	std_logic	Pin de Reset
i_start	std_logic	Inicia el calculo
i_U	std_logic_vector(31:0)	Tensión de Entrada de fase U
i_V	std_logic_vector(31:0)	Tensión de Entrada de fase V
i_W	std_logic_vector(31:0)	Tensión de Entrada de fase W
o_valido	std_logic	Indica que las salidas son validas
o_alfa	std_logic	Componente alfa de la transformada de Clarke
o_beta	std_logic	Componente beta de la transformada de Clarke

Tabla 5. Descripción de puertos

Este bloque se encargará de realizar la transformada de Clarke, para obtener las representaciones bilineales de las señales trifasicas.

La transformada de Clarke se define de la siguiente manera:

$$\begin{bmatrix} V_{\alpha} \\ V_{\beta} \end{bmatrix} = \frac{2}{3} * \begin{bmatrix} 1 & 0 & 1 \\ -\frac{1}{2} & \frac{\sqrt{3}}{2} & 1 \\ -\frac{1}{2} & -\frac{\sqrt{3}}{2} & 1 \end{bmatrix} * \begin{bmatrix} V_U \\ V_V \\ V_W \end{bmatrix} \quad (28)$$

El producto por la constante $\frac{2}{3}$ es para mantener la amplitud de las señales, ya que el producto matricial por si solo, hace que las amplitudes se reduzcan.

Cabe aclarar que el calculo de esta transformada se realizara en cada pulso del trigger de salida del Modulador, el cual indica que comienza un ciclo de PWM, y se utiliza dicho ciclo para todo el proceso de calculo. Asi se asegura que los valores de α y β se actualizan en cada ciclo, y se pueden usar para el calculo de los PWM del siguiente ciclo.

3.7. Generador de referencias angulares

Se diseñaron dos formas de generar las referencias angulares, una es mediante un simple bloque que genera un diente de sierra sincronizado con el cruce en cero del valor α de la transformada de Clarke, y la otra es mediante el calculo trigonométrico del $\arctan\left(\frac{\beta}{\alpha}\right)$, que da el valor del ángulo del vector rotante.

A continuación se mostraran ambos bloques, que a pesar de tener diferentes formas de generar las referencias, el resultado final fue el mismo. La gran diferencia entre ellos es que el bloque Generador Diente de Sierra no es sintetizable, ya que internamente se utilizan variables de tipo real, y ademas se le deben definir mediante generics la frecuencia de la señal de entrada y salida, lo que hace que no sea un bloque reutilizable.

3.7.1. Generador Diente de Sierra



Figura 8. Bloque Generador Diente de Sierra

Nombre	Tipo	Descripción
i_clk	std_logic	Señal de Reloj de 100KHz
i_rst	std_logic	Pin de Reset
i_sin	std_logic_vector(31:0)	Señal α proveniente de la T. Clarke
o_angle	std_logic_vector(31:0)	Diente de sierra que representa el angulo

Tabla 6. Descripción de puertos

Este bloque se encargara de generar un diente de sierra que representa el angulo del vector rotante generado por la trifasica, que para ellos se utilizara como entrada el valor de α proveniente de la transformada de Clarke.

Los valores de la diente de sierra irán desde 0 hasta 2048 (11 bits), que es el valor máximo que pueden tener las entradas α y β del Modulador.

3.7.2. CORDIC - atan2

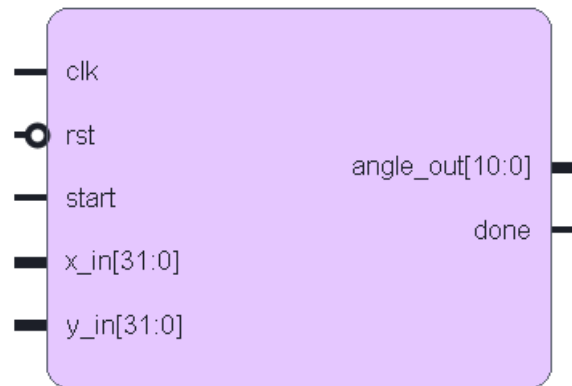


Figura 9. Bloque CORDIC - atan2

Nombre	Tipo	Descripción
i_clk	std_logic	Señal de Reloj de 100KHz
i_rst	std_logic	Pin de Reset
start	std_logic	Señal de inicio del cálculo
x_in	std_logic_vector(31:0)	Señal de entrada x
y_in	std_logic_vector(31:0)	Señal de entrada y
angle_out	std_logic_vector(10:0)	Resultado de la operación atan2
done	std_logic	Señal de finalización del cálculo

Tabla 7. Descripción de puertos

Este bloque calcula el angulo del vector rotante realizando el calculo

$$\arctan 2 \left(\frac{\beta}{\alpha} \right) \quad (29)$$

mediante el algoritmo CORDIC, que es un algoritmo iterativo que permite calcular funciones trigonométricas de manera eficiente.[3]

En este caso, la LUTs que se utiliza, van a ser valores de $\arctan(2^{-i})$ para cada iteración i, y

el numero de iteraciones se definió como 20, para obtener una buena precisión en el resultado. Esta cantidad hace que el bloque tenga una latencia de 20 ciclos de reloj, por lo que se le agregó una señal de salida "done" para indicar cuando el resultado es valido, y así sincronizar el flujo de calculo del sistema.

La principal razón por la que se eligió la función trigonométrica `atan2` y no `atan`, es que **`atan2` tiene en cuenta el signo de las entradas para determinar el cuadrante del resultado**, lo que es importante para obtener un resultado correcto en todo el rango de entrada. Mientras que `atan` solo devuelve un resultado entre -90° y 90° , limitándose a dos cuadrantes, lo que no es suficiente para representar el ángulo completo del vector rotante.

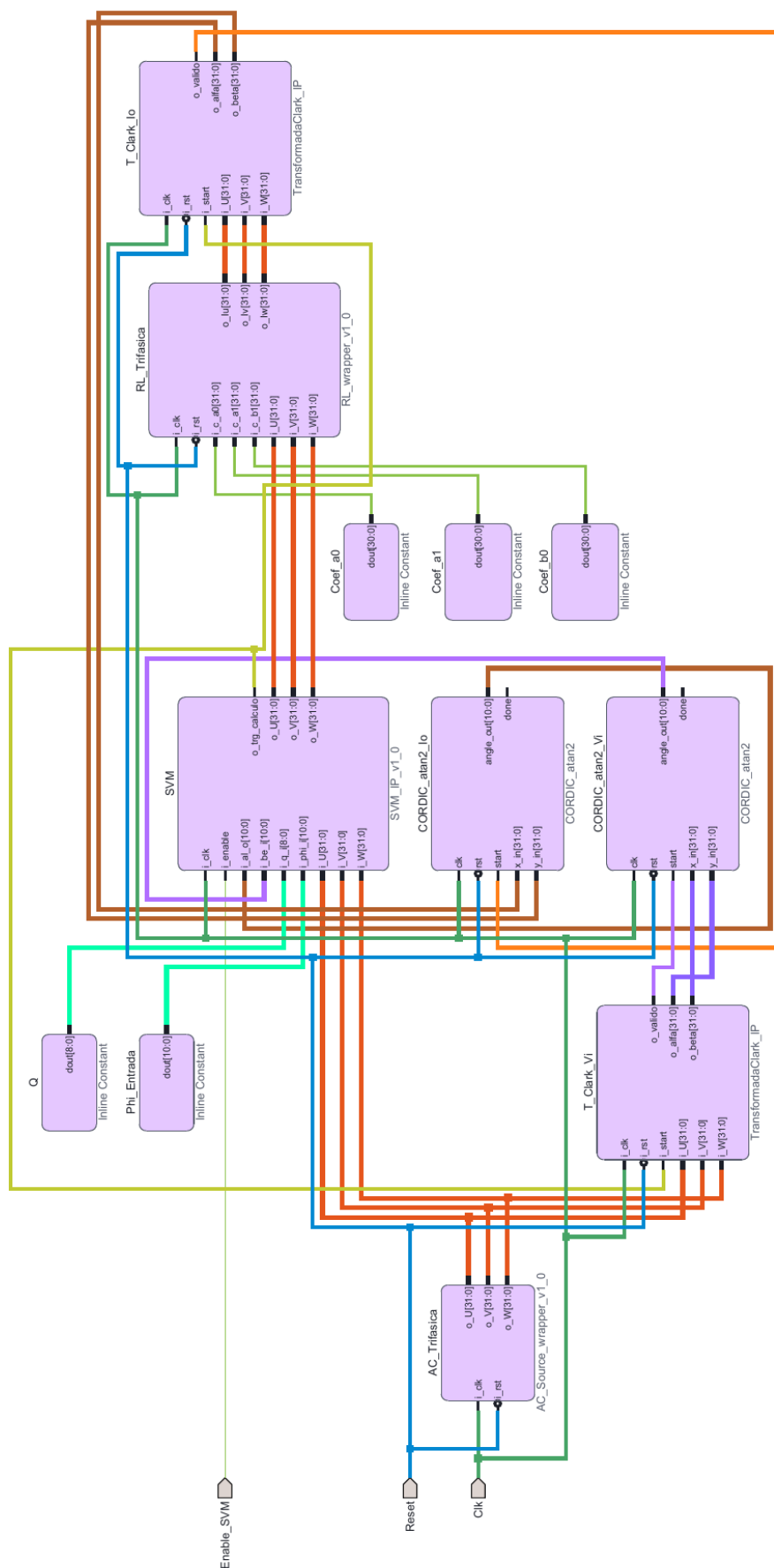


Figura 10. Diagrama de bloque del sistema

4. Simulaciones

4.1. Fuente de señal trifasica y Transformada de Clarke

Se simulo la fuente de señal trifasica con un clock de 100MHz, en conjunto con el bloque encargado de realizar la transformada de Clarke y se obtuvieron las siguientes señales:

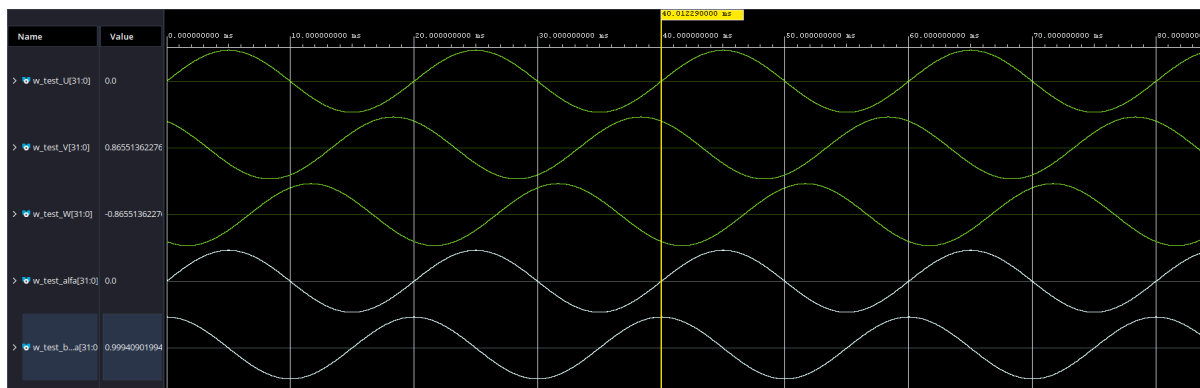


Figura 11. Salida de la Fuente trifasica + Transformada de Clarke

Las 3 señales superiores (w_test_*) son la entrada trifasica, y las 2 inferiores (w_test_alpha y w_test_beta) son las señales α y β respectivamente.

La amplitud de la fuente trifasica es de 1V, y al transformarse no cambia gracias al producto que se comento en la definición del bloque de la transformada de Clarke.

4.2. Generador Alfa y Beta

4.2.1. Generador Diente de Sierra

Utilizando la salida α de la transformada de Clarke, se simulo el bloque encargado de generar la diente de sierra que puede representar los ángulos que necesita el modulador, y se obtuvo la siguiente salida:

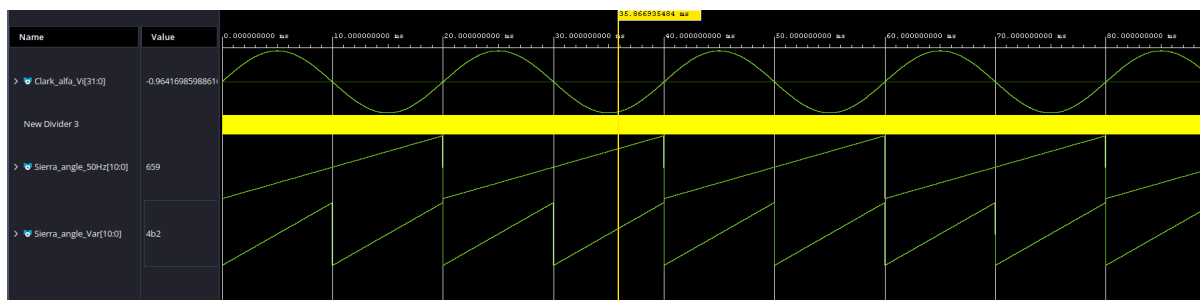


Figura 12. Salida del generador Diente de Sierra

La señal senoidal superior es la señal α proveniente de la transformada de Clarke, y las dos inferiores son señales diente de sierra de 50Hz (Sierra_angle_50Hz) y 100Hz (Sierra_angle_Var).

Cada vez que la diente de sierra se reinicia a cero, es el equivalente a iniciar una nueva vuelta en la rotación del vector formado por la trifasica.

4.2.2. CORDIC - atan2

Utilizando las señales α , β y dato valido del bloque encargado de realizar transformada de Clarke como entradas, se simulo el bloque encargado de calcular el atan2 mediante el algoritmo CORDIC.

$$x_{in} = \beta \quad (30)$$

$$y_{in} = \alpha \quad (31)$$

Se obtuvo la siguiente salida:

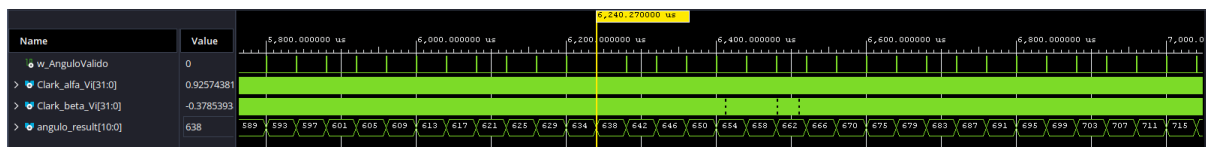


Figura 13. Salida del bloque CORDIC - atan2

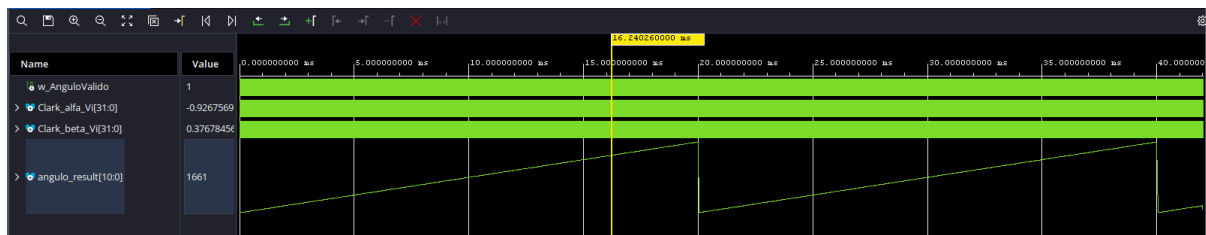


Figura 14. Salida del bloque CORDIC - atan2 (Vista con diente de sierra)

Una ventaja que tiene este bloque es que no es necesario indicarle al componente la frecuencia de la señal de referencia, ni de que frecuencia se desea que sea la, ya que la salida es el resultado de una función trigonométrica, por lo que la salida depende unicamente de las entradas en el instante que se indique con la entrada start.

4.3. Carga RL trifasica

Se simulo la carga RL trifasica con un clock de 100MHz y como entrada se utiliza la fuente de señal anteriormente probada.

El calculo de los coeficientes del IIR, se selecciono los siguientes valores:

- R: 1.2Ω
- L: 0.012 H
- Fs: 100 MHz

Se obtuvo la siguiente salida:

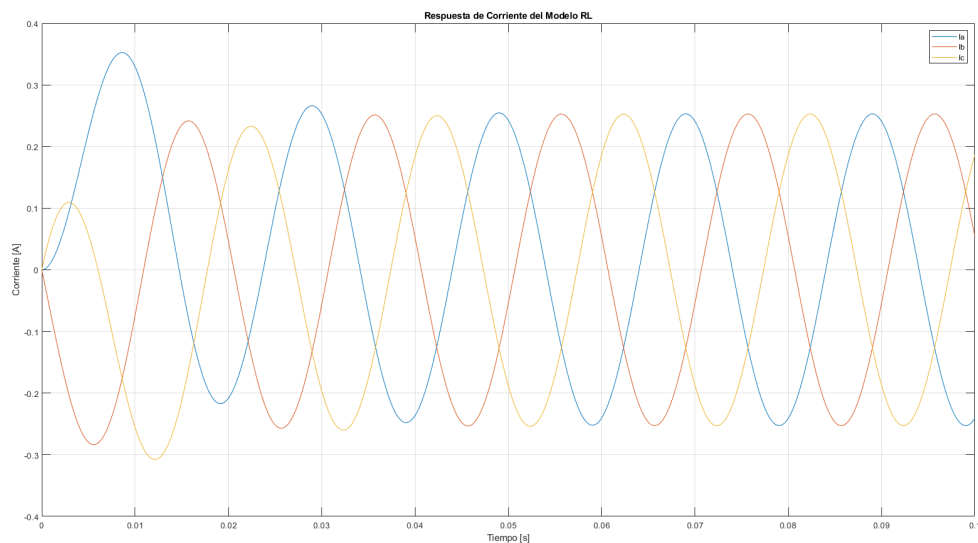


Figura 15. Respuesta de la carga RL trifasica

Se observa que aproximadamente en los primeros 30ms hay un transitorio, luego de este tiempo las corrientes llegan a un estado estable.

4.4. Modulador SVM

Una vez realizadas las simulaciones en los bloques individuales, se simuló el sistema completo de la figura .

Para la obtención de las corrientes en la carga, se exporta los valores de las señales de salida del modulador SVM a python, y se simula la respuesta de la carga RL trifasica con los mismos parámetros. Esto es debido a que la simulación en VHDL de todo el sistema completo, es muy lenta y consume muchos recursos computacionales.

Para el cálculo de β_i y α_o se utiliza la misma señal trifasica (de tensión de entrada) que se utiliza para alimentar al resto del sistema.

Modificando la declaración de los generadores de diente de sierra se pueden realizar simulaciones con diferentes frecuencias de la señal de referencia. Para este caso en ambos generadores se selecciono una frecuencia de 50Hz.

Se obtuvo la siguiente salida de corriente en la carga:

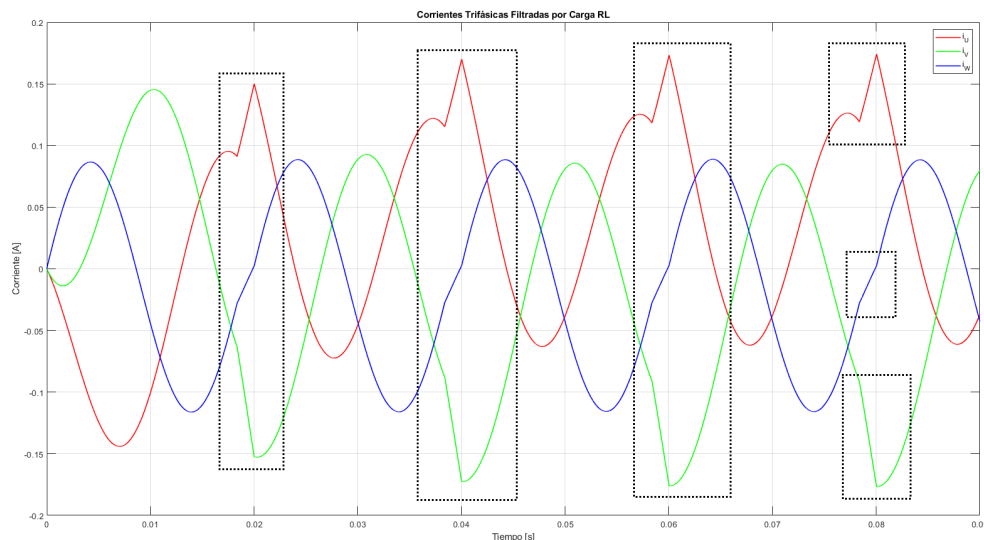


Figura 16. Respuesta de la carga RL trifasica con modulador SVM

Se observa claramente que la señal de corriente posee una discontinuidad, los valores que deberían seguir una senoidal se comportan con una función lineal desplazando todo en el eje vertical y de forma periódica antes de completar un periodo.

Para tener un vistazo un poco mas profundo sobre que podría estar pasando, se buco analizar las señales conmutadas que salen del modulador SVM, y se obtuvo lo siguiente:

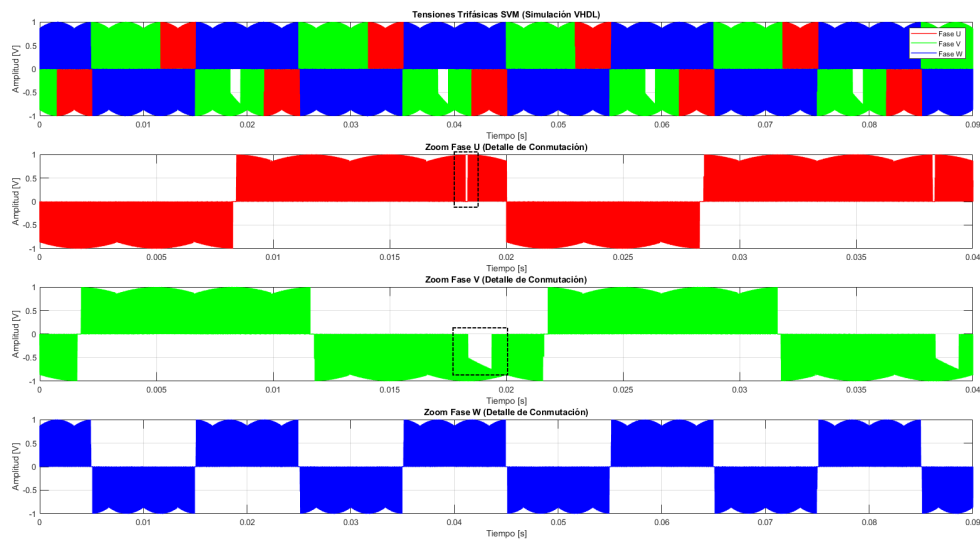


Figura 17. Señales conmutadas del modulador SVM

Las partes recuadradas son vectores nulos, el problema esta en que no se observa una simetría en la secuencia de las conmutaciones, lo cual es un indicio de que el problema puede estar interno en el Modulador, un error numérico al momento de calcular los tiempos de los PWM de las combinaciones.

4.4.1. Analisis interno

Al analizar las señales internas del componente, se observa que se levanta una señal flag (hi_bit) en el instante en que se corta la senoidal en la simulación y se reinicia a cero al momento de comenzar el siguiente periodo de la señal de salida (Comienza un nuevo calculo de las secuencias), viendo el código, se encontró que esta flag se activa cuando el producto de los valores de cosenos por los valores de las señales α y β son mayores a un valor umbral.



Figura 18. Señales internas del modulador SVM

En la representación de los valores del SVM, se eligió el formato complemento a 2, para representar valores negativos y positivos (La tabla de cosenos esta hecha con esta lógica). Analizando el archivo, se observo que unicamente se esta utilizando la librería

ieee.std_logic_unsigned.all, produciendo que todos los valores sean interpretados como números sin signo (unsigned), por lo que si el valor de $\cos(x)$ es negativo, se interpreta como un número positivo de alto valor, ya que los números negativos en complemento a 2 se "llenan" de valores 1 en las posiciones mas altas, causando errores en los cálculos o sobrecargas al momento de realizar un producto (que es lo que esta ocurriendo).

4.4.2. Solución

Conocido el problema, la solución que se planteo es mantener la librería **ieee.std_logic_unsigned.all** para no alterar el funcionamiento de la mayoría del código, especialmente las maquinas de estado, y para los cálculos que involucren números negativos se agrego el llamado a la librería **ieee.std_logic_arith.all**, la cual permite realizar operaciones con números con signo (signed), con funciones que agrega.

De esta forma, se modificaron las líneas de código donde se realizan los productos de los valores de cosenos de las señales α y β , utilizando la función `signed()` para no cometer el error de mal formateo de valores.

Luego de realizar estos cambios, se volvió a simular el sistema completo y se obtuvo la siguiente salida:

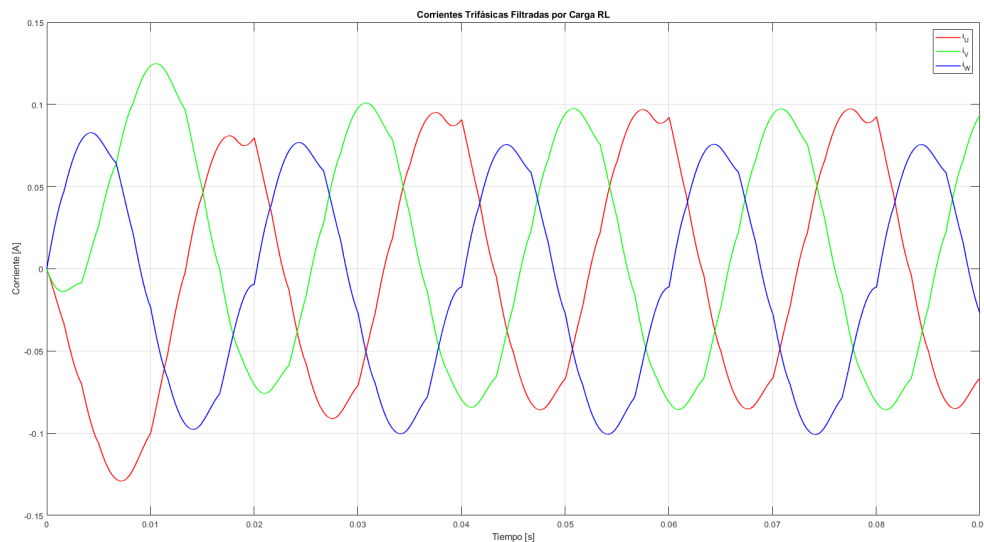


Figura 19. Respuesta de la carga RL trifasica con modulador SVM corregido

y como curiosidad se puede ver la trayectoria del vector rotante generado por las corrientes en la carga:

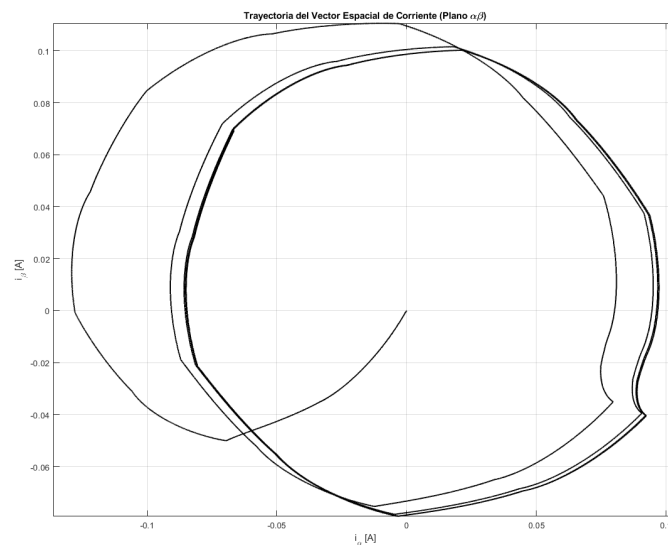


Figura 20. Vector rotante de las corrientes en la carga RL trifasica

En ambas imágenes se observa que existe un transitorio, pero luego de este las señales llegan a un estado estable.

Todavía existe una leve esta discontinuidad y un leve offset en la fase W ($\approx 0,05A$), que se deben a una falta de precisión dentro de los cosenos del SVM y los cálculos internos, además que, el sistema de simulación en python no es perfecto y tiene un error numérico al momento de simular la respuesta de la carga RL con señales PWM, pero es mucho menor al error que se tenía anteriormente.

El Modulador presente en el repositorio de GitHub ya posee esta solución implementada

5. Implementación en FPGA

Antes de pasar el diseño a la FPGA, se deben realizar una serie de pasos:

- **Síntesis:** Es el proceso de traducir el código VHDL a una representación de bajo nivel que describe las compuertas lógicas y flip-flops necesarios para implementar el diseño.
- **Implementación:** Es el proceso de asignar las compuertas lógicas y flip-flops a los recursos físicos del FPGA, incluyendo la colocación y enrutamiento.
- **Programación:** Es el proceso de cargar el diseño implementado en el FPGA para su ejecución.

La síntesis y la implementación son procesos críticos que pueden revelar problemas de timing, es decir, que algunas señales no cumplen con los requisitos de tiempo establecidos. Estos problemas pueden ser causados por la complejidad del diseño, la cantidad de operaciones combinacionales en una ruta crítica, o la falta de recursos en el FPGA. Cuando se completa, se puede observar como el Vivado traduce el código VHDL en las compuertas lógicas y flip-flops que componen el diseño.

Es importante abordar estos problemas de timing para garantizar que el diseño funcione correctamente a la frecuencia deseada.

5.1. Síntesis

La síntesis se realizaba cada vez que se realizaba una simulación, así que esta etapa se realizaba de forma continua durante el desarrollo del proyecto.

5.2. Implementación

Cuando se realizó el primer proceso de implementación, se observaron varios errores de timing, es decir, que algunas señales no cumplían con los requisitos de tiempo establecidos.

5.2.1. Problemas de Timing

El problema se origina en un fallo crítico de Setup Time en el componente base de la Carga RL trifásica, RL Fase.

El WNS (Worst Negative Slack) es de -11.158ns @ 100MHz. Esto quiere decir que la ruta crítica excede los 21ns, indicando que la operación requiere más de 2 ciclos de reloj para completarse. Esto se debe a que el bloque contiene varias operaciones de multiplicación y suma en cascada, que se intentan realizar en un único ciclo de clock.

5.2.2. Soluciones

Se realizaron una serie de modificaciones para intentar solucionar el problema de timing que aplican tanto al componente RL Fase, como al diseño general:

- **Pipelining:**
 - **Implementación:**
 - **Etapas 1:** Se calculan los productos, haciendo que la síntesis utilice los bloques DSP del FPGA.
 - **Etapas 2:** Suma de productos y actualización de estados.

- **Resultado:** El WNS mejoro de -11.158ns a -4.623ns. La suma de términos de 64 bits mas un resize sigue siendo demasiado lenta para la FPGA.
- **Optimización Lógica:**
 - **Estrategia:** Reducir la complejidad combinacional del bloque.
 - **Simplificación Aritmética:** Mediante parámetros en la función `resize`, se sustituyo el tipo de saturación intermedia, para reducir la cantidad de comparadores, dejando la saturación solo en la etapa final.
 - **Resultado:** El WNS mejoro de -4.623ns a -1.823ns. Pero sigue siendo insuficiente.
- **Diseño Multi-Clock (Dual Domain):**
 - **Estrategia:** Debido a que todavía se requerían unos nanosegundos más para cumplir con el timing, se decidió dividir el diseño en dos dominios de reloj. Basándonos en que el sistema (Carga RL) no es rápido (del orden de kHz), se puede permitir que el bloque funcione a una frecuencia menor. Entonces se decidió que la parte aritmética funcione a 50MHz, mientras que lo relacionado a la comunicación AXI4-Lite se mantenga a 100MHz.
 - **Resultado:** El WNS mejoro de -1.823ns a WNS >0, cumpliendo finalmente con los requisitos de timing.

5.2.3. Conclusion General

El problema del timing se resolvió desde varios enfoques: **Pipelining**, **Optimización Lógica** y **Diseño Multi-Clock**. Para así poder garantizar el modelo matemático, sin comprometer la velocidad de comunicación con el PS.

6. Comentarios Finales

6.1. Conclusiones

6.1.1. Validación de la Metodología HIL (Hardware In Loop)

- Se mostró que realizar una metodología HIL de este tipo de sistemas de potencia es posible, y viable. Mostrando un equilibrio entre la complejidad de la implementación y los beneficios obtenidos, como lo es la reducción de costos y riesgos asociados a la alta potencia.

6.1.2. Desafíos en Aritmética de Punto Fijo y VHDL

- El diseño y programación en FPGA es un paradigma completamente diferente a la programación tradicional, se tuvo que tomar decisiones de diseño que no se hubieran tomado en un lenguaje de software como C, como lo es el manejo de tipos de datos, la forma de interpretar variables reales y la forma de realizar operaciones aritméticas.

En este proyecto se planteó una base de formato de representación de variables y de ahí se fue aprendiendo a manejarlo, además de interpretar los números en complemento a 2, que como se vio con el SVM, de ser mal utilizado puede generar errores significativos en el resultado final.

La elección del formato de punto fijo Q8.24 demostró ser adecuada para mantener un equilibrio entre el rango (representando valores en $[-128; 128]$) y la resolución necesaria para los cálculos del sistema.

6.1.3. Implementación en FPGA y Análisis de Timing

- Se descubrió que la implementación física en FPGA introduce desafíos adicionales relacionados con los tiempos de propagación de las señales, especialmente cuando se realizan operaciones aritméticas complejas.

Gracias a eso, se aprendieron nuevas técnicas para poder reducir este error, como lo son el *pipelining*, la optimización lógica y la arquitectura de múltiples relojes.

6.1.4. Arquitectura y Herramientas

- El diseño estructural por componentes (módulos) permitió una mejor organización del proyecto, facilitando la identificación y corrección de errores.

La utilización de Python para generar las LUTs de la señal seno y validar datos se mostró como una herramienta valiosa para complementar el diseño en VHDL, permitiendo una configuración más rápida y precisa.

6.1.5. Conclusion Final

El proyecto terminó exitosamente con la obtención de corrientes trifásicas sinusoidales filtradas y un vector espacial de corriente rotante correcto. Si bien existieron desafíos significativos en cuanto a la integridad de la señal (signos) y los tiempos de propagación (*timing*), las soluciones implementadas permitieron establecer una plataforma sólida para el testeado de algoritmos de control SVM.

6.2. Posibles Mejoras

Entre las posibles mejoras posibles para el proyecto se pueden mencionar:

- Implementar un sistema de control en tiempo real para ajustar dinámicamente los parámetros del SVM en función de las condiciones de operación. Esto podría hacerse tanto dentro de la misma FPGA, o utilizando el microcontrolador presente en la placa de desarrollo.

Lo mas recomendable seria dentro de la misma FPGA, para evitar latencias asociadas a la comunicación entre el microcontrolador y la FPGA.

- Analizar el impacto de la resolución de los datos en el rendimiento del SVM, experimentando con diferentes formatos de punto fijo o incluso con aritmética de punto flotante, para evaluar si se pueden obtener mejoras significativas en la precisión y estabilidad del sistema.
- Ampliar el proyecto para incluir la simulación de fallas en el sistema, como cortocircuitos o fallas a tierra, y evaluar cómo el SVM responde a estas condiciones. Esto podría ayudar a mejorar la robustez del sistema y su capacidad para manejar situaciones imprevistas.
- Eliminar los modelos de la fuente trifasica y la carga RL, reemplazarlos por mediciones mediante ADCs y sensores de corriente/voltaje, para obtener datos en tiempo real y validar el rendimiento del SVM en condiciones de operación reales.

Teniendo por fuera la fuente y la carga, se podría realizar pruebas y análisis de la carga armónica que genera el SVM sobre la red eléctrica, y así evaluar su eficiencia.

- Tener alguna forma de monitoreo y configuración de los SetPoints del control por fuera de la FPGA, ya sea a través de una interfaz de usuario en el microcontrolador o mediante una conexión, Ethernet por ejemplo.

A. Conversor Matricial - SVM

La modulación por vector espacial (SVM, por sus siglas en inglés) representa una técnica avanzada y fundamental para el control de convertidores matriciales trifásico. Su importancia radica en su capacidad de control total tanto del vector de tensión de salida como del ángulo de desplazamiento instantáneo de la corriente de entrada. Entre sus ventajas clave se encuentran la posibilidad de alcanzar la máxima relación de transferencia de tensión posible y controlar el factor de potencia de entrada.[1]

A.1. Principio Fundamental del Algoritmo SVM

El concepto principal del algoritmo SVM se basa en la selección y aplicación de configuraciones de conmutación específicas durante intervalos de tiempo calculados con precisión dentro de un período de ciclo T_s . De las 27 combinaciones de interruptores posibles, 21 se consideran útiles para el algoritmo SVM:

Switching configuration list	Switches On			v_o	α_o	i_i	β_i
+1	S_{11}	S_{22}	S_{32}	$2/3 v_{12i}$	0	$2/\sqrt{3} i_{o1}$	$-\pi/6$
-1	S_{12}	S_{21}	S_{31}	$-2/3 v_{12i}$	0	$-2/\sqrt{3} i_{o1}$	$-\pi/6$
+2	S_{12}	S_{23}	S_{33}	$2/3 v_{23i}$	0	$2/\sqrt{3} i_{o1}$	$\pi/2$
-2	S_{13}	S_{22}	S_{32}	$-2/3 v_{23i}$	0	$-2/\sqrt{3} i_{o1}$	$\pi/2$
+3	S_{13}	S_{21}	S_{31}	$2/3 v_{31i}$	0	$2/\sqrt{3} i_{o1}$	$7\pi/6$
-3	S_{11}	S_{23}	S_{33}	$-2/3 v_{31i}$	0	$-2/\sqrt{3} i_{o1}$	$7\pi/6$
+4	S_{12}	S_{21}	S_{32}	$2/3 v_{12i}$	$2\pi/3$	$2/\sqrt{3} i_{o2}$	$-\pi/6$
-4	S_{11}	S_{22}	S_{31}	$-2/3 v_{12i}$	$2\pi/3$	$-2/\sqrt{3} i_{o2}$	$-\pi/6$
+5	S_{13}	S_{22}	S_{33}	$2/3 v_{23i}$	$2\pi/3$	$2/\sqrt{3} i_{o2}$	$\pi/2$
-5	S_{12}	S_{23}	S_{32}	$-2/3 v_{23i}$	$2\pi/3$	$-2/\sqrt{3} i_{o2}$	$\pi/2$
+6	S_{11}	S_{23}	S_{31}	$2/3 v_{31i}$	$2\pi/3$	$2/\sqrt{3} i_{o2}$	$7\pi/6$
-6	S_{13}	S_{21}	S_{33}	$-2/3 v_{31i}$	$2\pi/3$	$-2/\sqrt{3} i_{o2}$	$7\pi/6$
+7	S_{12}	S_{22}	S_{31}	$2/3 v_{12i}$	$4\pi/3$	$2/\sqrt{3} i_{o3}$	$-\pi/6$
-7	S_{11}	S_{21}	S_{32}	$-2/3 v_{12i}$	$4\pi/3$	$-2/\sqrt{3} i_{o3}$	$-\pi/6$
+8	S_{13}	S_{23}	S_{32}	$2/3 v_{23i}$	$4\pi/3$	$2/\sqrt{3} i_{o3}$	$\pi/2$
-8	S_{12}	S_{22}	S_{33}	$-2/3 v_{23i}$	$4\pi/3$	$-2/\sqrt{3} i_{o3}$	$\pi/2$
+9	S_{11}	S_{21}	S_{33}	$2/3 v_{31i}$	$4\pi/3$	$2/\sqrt{3} i_{o3}$	$7\pi/6$
-9	S_{13}	S_{23}	S_{31}	$-2/3 v_{31i}$	$4\pi/3$	$-2/\sqrt{3} i_{o3}$	$7\pi/6$
0 ₁	S_{11}	S_{21}	S_{31}	0	-	0	-
0 ₂	S_{12}	S_{22}	S_{32}	0	-	0	-
0 ₃	S_{13}	S_{23}	S_{33}	0	-	0	-

Figura 21. Configuraciones de Switching utilizadas en la SVM

- 18 configuraciones **Activas**: Generan un vector de tensión de salida y un vector de corriente de entrada con direcciones fijas y discretas.
- 3 configuraciones **Ceros**: Producen vectores de tensión de salida y corriente de entrada nulos.
- 6 configuraciones restantes que se descartan porque conectan cada fase de salida a una fase de entrada diferente, generando vectores de tensión y corriente con direcciones variables que no son útiles para la síntesis de vectores de referencia.

El objetivo del algoritmo es sintetizar un vector de tensión de salida de referencia (v_o) y controlar simultáneamente el ángulo de desplazamiento de la corriente de entrada (α_o). Esto se logra promediando en el tiempo el efecto de cuatro configuraciones activas seleccionadas y una o más configuraciones cero en cada ciclo.

A.2. Definición y Propósito de los Parámetros de Control

Para implementar el algoritmo SVM, dos parámetros angulares son fundamentales para determinar los ciclos de trabajo (Duty cycles) de las configuraciones de conmutación. Estos parámetros se definen de la siguiente manera:

- α_o : Corresponde al ángulo de fase del vector de tensión de salida, también medido con respecto a la línea que bisecta su sector correspondiente.
- β_i : Representa el ángulo de fase del vector de corriente de entrada, medido con respecto a la línea que bisecta el sector en el que se encuentra dicho vector en un instante determinado.

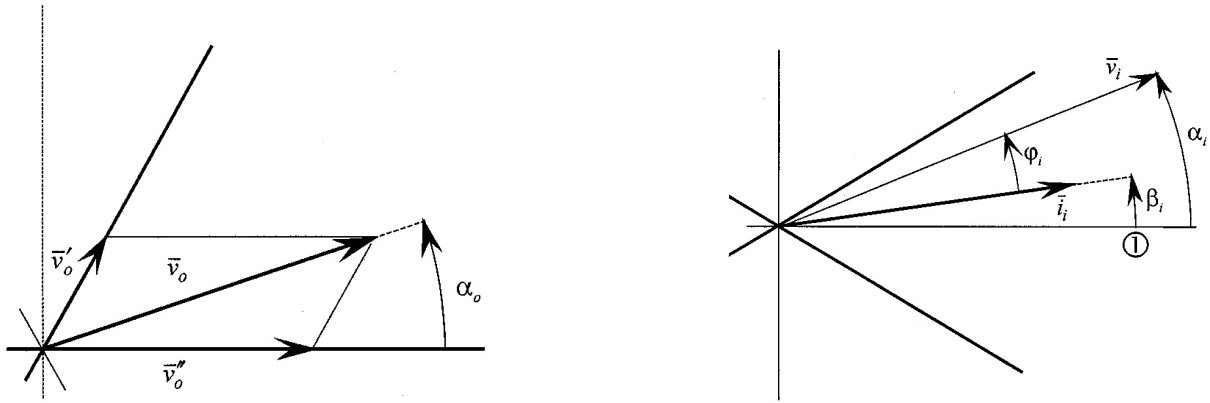


Figura 22. α_o , β_i y ϕ

Las ecuaciones del modelo son las siguientes:

$$\delta^I = (-1)^{K_v+K_i+1} \frac{2}{\sqrt{3}} q \frac{\cos(\tilde{\alpha}_o - \pi/3) \cos(\tilde{\beta}_i - \pi/3)}{\cos \varphi_i} \quad (32)$$

$$\delta^{II} = (-1)^{K_v+K_i} \frac{2}{\sqrt{3}} q \frac{\cos(\tilde{\alpha}_o - \pi/3) \cos(\tilde{\beta}_i + \pi/3)}{\cos \varphi_i} \quad (33)$$

$$\delta^{III} = (-1)^{K_v+K_i+1} \frac{2}{\sqrt{3}} q \frac{\cos(\tilde{\alpha}_o + \pi/3) \cos(\tilde{\beta}_i - \pi/3)}{\cos \varphi_i} \quad (34)$$

$$\delta^{IV} = (-1)^{K_v+K_i+1} \frac{2}{\sqrt{3}} q \frac{\cos(\tilde{\alpha}_o + \pi/3) \cos(\tilde{\beta}_i + \pi/3)}{\cos \varphi_i} \quad (35)$$

Siendo $\tilde{\alpha}_o$ y $\tilde{\beta}_i$ los valores de α_o y β_i medidos desde la línea bisecta del sector correspondiente:

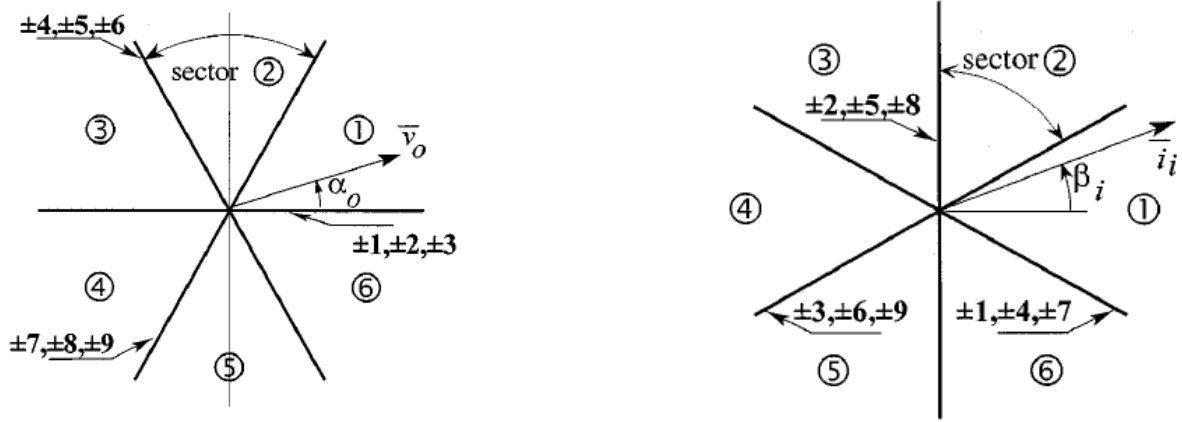


Figura 23. División en sectores de los ángulos

La función de α_o y β_i es crucial, ya que actúan como las entradas directas a las ecuaciones (1-4) del modelo. Estas ecuaciones calculan los ciclos de trabajo de las cuatro configuraciones de conmutación activas necesarias para generar la salida deseada en cada instante, traduciendo los objetivos de control en tiempos de aplicación concretos para los interruptores.

Se puede hacer un análisis adicional, para tener una estrategia de control mas factible, la suma de los valores absolutos de los cuatro Duty Cycles debe ser menor o igual a la unidad.

$$|\delta^I| + |\delta^{II}| + |\delta^{III}| + |\delta^{IV}| \leq 1 \quad (36)$$

Las configuraciones Ceros se utilizan para completar el ciclo del periodo.

Uniendo las ecuaciones (32-35) dentro de 36 y luego de unos manejos algebraicos, se obtiene:

$$q \leq \frac{\sqrt{3}}{2} \frac{|\cos\varphi_i|}{\cos\tilde{\beta}_i \cos\tilde{\alpha}_o} \quad (37)$$

Esta ecuación representa, en cualquier instante, el factor teórico máximo de transferencia de voltaje, el cual es en función de los ángulos de la tensión de salida, y la corriente de entrada y el $\cos\varphi_i$ de la entrada. Es útil ver que en el caso de que la tensión de entrada y salida sena balanceadas, el máximo factor de voltaje de transferencia ocurre cuando $\tilde{\beta}_i$ y $\tilde{\alpha}_o$ son iguales a 1, teniendo como resultado:

$$q \leq \frac{\sqrt{3}}{2} |\cos\varphi_i| \quad (38)$$

Y luego, asumiendo factor de potencia 1, el máximo factor de transferencia de voltaje es de 0.866.

El proceso de modulación SVM puede resumirse en los siguientes pasos, que se repiten en cada período de ciclo:

1. **Selección:** Se identifican los sectores actuales en los que se encuentran el vector de tensión de entrada de referencia y el vector de corriente de salida deseado. Con base en esta combinación de sectores, se seleccionan cuatro configuraciones de conmutación activas específicas.
2. **Calculo:** Se calculan los ciclos de trabajo correspondientes a cada una de estas cuatro configuraciones (δ^I , δ^{II} , δ^{III} , δ^{IV}) utilizando los ángulos α_o y β_i como entradas en las ecuaciones generales de modulación (1-4).
3. **Aplicación:** Las cuatro configuraciones activas se aplican secuencialmente durante los intervalos de tiempo definidos por sus respectivos ciclos de trabajo. Para minimizar los armónicos y optimizar el rendimiento, estas configuraciones se suelen aplicar siguiendo un patrón de conmutación simétrico de doble lado (double-sided switching pattern), donde la secuencia se aplica en la primera mitad del ciclo y se invierte en la segunda.
4. **Completado:** El tiempo restante del período del ciclo se completa mediante la aplicación de una o más de las tres configuraciones cero.

Este uso de las configuraciones cero introduce dos grados de libertad intrínsecos en la modulación, un concepto cuya importancia fundamental es revelada y formalizada por el enfoque DCSV.

Referencias

- [1] Domenico Casadei et al. «Matrix Converter Modulation Strategies: A New General Approach Based on Space-Vector Representation of the Switch State». En: *IEEE Transactions on Industrial Electronics* 49.2 (2002), págs. 370-381.
- [2] UTN FRH ASE. *NCO + LUT*. 2024. URL: https://gitlab.com/educiaafpga/ejemplos/-/tree/master/Ejemplos_Base/19-NCO_LUT.
- [3] Dr. Steve Arar. *An Introduction to the CORDIC Algorithm*. 2017. URL: <https://www.allaboutcircuits.com/technical-articles/an-introduction-to-the-cordic-algorithm/>.